VIDEO SCRIPTS & RECORDING GUIDE — LAKSH AHUJA (24BDS0212)

AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry

> *How to use: for each review, prepare everything in "Before you record", follow the step table while recording, and read the script aloud at a relaxed pace (about 135 words per minute). Lines marked ▶ are screen cues, not spoken. Fill in your team number.*

Recording Setup (all three videos)

- **Recorder:** OBS Studio (free) with two sources — the screen, and your webcam as a small box in a corner — so your face is visible the whole time. A phone on a stand filming you beside the laptop also works.
- **Length:** each script is about 5 minutes when read at a relaxed pace. The rules allow 5–7 minutes, so you have room to pause while something loads.
- **First 15 seconds:** face on camera, say your name, register number, team number and the project title.
- **Before pressing record:** close WhatsApp and notifications, set the browser zoom to 125 %, use a large terminal font, and keep this script open on a phone or second screen.
- **Live parts:** record demos in one take without cuts; if something fails, say what happened and restart it on camera — that counts as honest evidence.
- **File names:** LakshAhuja_R1_Video.mp4, LakshAhuja_R2_Video.mp4, LakshAhuja_R3_Video.mp4.

1. Review 1 — Problem Identification & Proposal

Date: 24/8/2026 **Spoken length:** 653 words ≈ 4.8 minutes **File name:** LakshAhuja_R1_Video.mp4

Before you record

1. Open LakshAhuja_R1.pdf (your R1 report) in a PDF viewer.
2. Open one cyclone satellite image in a browser tab (for example from IMD's or MOSDAC's website).
3. Optional: put Figure 1 (block diagram), the literature table and the timeline on 4–5 simple slides; scrolling the PDF is also fine.

What to show, step by step

Time	On screen	What you do
0:00–0:15	Webcam full screen, then the report cover page	Introduce yourself (name, register no., team no., title).
0:15–1:30	Report section 1 (problem) and Table 1 (evidence); the satellite image tab	Read the problem statement; point at the IMD row, the Esposito row and the 29-minute calculation.
1:30–2:45	Table 2 (literature survey)	Scroll slowly through the 15 papers; stop on rows 10–12 (URLLC, age of information) when you explain the gap.
2:45–4:15	Figure 1 (block diagram), then the feasibility and risk tables	Trace the data path with the mouse: sensors → gateway → QoS → server → dashboard. Point at your yellow blocks.
4:15–5:00	Team role table and weekly timeline	Point at your row and the three weeks; close with ‘Thank you’ on webcam.

Script

0:00–0:15 Introduction

Hello, I am Laksh Ahuja, register number 24BDS0212, team number ____. Our project is “AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry”, and in this video I present my individual work for Review 1.

> *SHOW: your face, then the report cover.*

0:15–1:30 Problem and evidence

Our problem is this: when a cyclone approaches, ocean buoys and coastal stations have to send their readings over slow and unreliable links, and the readings that matter most get delayed or lost exactly when the weather is worst.

Why does it matter? According to the India Meteorological Department, India has a coastline of about seven and a half thousand kilometres, and the 1999 Odisha super cyclone alone killed about ten thousand people. The people affected are coastal communities, fishers, and the disaster-management officials who order evacuations.

A 2022 review of IoT early-warning systems by Esposito and colleagues found gaps in current designs and recommends adding an edge layer close to the sensors. That supports our approach.

My own calculation shows the bandwidth side of the problem: one four-band satellite image is about two megabytes. On a nine-point-six kilobit satellite link, that single image takes about twenty-nine minutes to send — and any alarm queued behind it waits that long.

> *SHOW: Table 1 (evidence) and a cyclone satellite image.*

1:30–2:45 Literature survey and gap

I surveyed fifteen papers from 2018 to 2022. Edge-computing surveys, such as Yu, Abbas and Pham, show that processing near the data cuts delay and backhaul load, but they do not rank traffic by how dangerous an event is.

Work on ultra-reliable low-latency communication, by Bennis and Popovski, and the age-of-information survey by Yates show how to protect urgent and fresh data — but they assume 5G infrastructure that a buoy at sea does not have.

Satellite IoT surveys show that remote links are slow, and Chen's review of machine learning for cyclone forecasting assumes the data has already arrived.

So the gap is: nobody ties link priority to the live cyclone risk computed at the server, while also repairing missing readings and showing what was estimated. That link between risk and the network is exactly my part of the project.

> *SHOW: Table 2 (15 papers); stop on rows 10 to 12.*

2:45–4:15 Proposed solution and feasibility

Our solution is a coding and modelling project: a three-tier edge–cloud system. Sensor nodes send small packets to an edge gateway. The gateway labels them by importance, and the most important ones go first over the slow link to a central prediction server, which repairs missing data, calculates a cyclone risk score and shows it on a live dashboard.

We chose coding and simulation first, not fabrication, because we cannot put buoys in the ocean. Software lets us measure behaviour under congestion and loss and repeat every experiment. Low-cost ESP32 nodes can be added later without changing the design.

> *SHOW: Figure 1, the block diagram — my blocks are highlighted.*

My blocks are the central server, the dashboard, the priority scheduling on the slow link, and the network emulation in Cisco Packet Tracer. My objectives are measurable: detect every missing sequence number, answer a prediction request in under one second, keep critical latency well below normal traffic under load, and refresh the dashboard at least every two seconds.

On feasibility: I already know Python, sockets and basic networking. I need to learn the FastAPI framework, queueing theory for the scheduler, OSPF routing in Packet Tracer, and automated testing. All tools are free. The main risks are timing bugs between threads, no real buoy data, and scope creep, and the risk table shows how I will handle each.

4:15–5:00 My role and timeline

In our team, Anvay builds the edge gateway and edge AI, Sreehari builds the sensors, hardware and data recovery, and I own everything after the gateway: scheduling, the server, the dashboard, the simulation and the tests.

My plan to Review 2: week one, agree the record format and build the Packet Tracer topology; week two, the server skeleton with the UDP listener and risk score; week three, the REST API, the first dashboard and an end-to-end test with my teammates' modules. Thank you.

> *SHOW: the team role table and the weekly timeline.*

2. Review 2 — Design, Development & Progress

Date: 14/9/2026 **Spoken length:** 586 words ≈ 4.3 minutes **File name:** LakshAhuja_R2_Video.mp4

Before you record

1. Get the Review 2 code (it is kept in the repository at commit 82bcb8a) in a separate folder: `git worktree add ../r2-prototype 82bcb8a` then `cd ../r2-prototype` and `pip install fastapi uvicorn`.
2. Arrange four terminal windows: server, gateway, two sensors.
3. Open LakshAhuja_R2.pdf, the GitHub page of the repository, and Cisco Packet Tracer with the team topology.
4. Do one practice run so the ports are free and nothing asks for firewall permission during recording.

What to show, step by step

Time	On screen	What you do
0:00–0:15	Webcam, then Figure 1 (architecture)	Introduce yourself.
0:15–1:55	Figure 1, Figure 2 (server flowchart), the parameter table, then central_server.py in an editor	Explain the three layers, then walk through your flowchart box by box; show the startup event and predict() in the code; then the comparison table (Table 3) for your design choices.
1:55–3:35	Terminals → browser	Terminal 1: `uvicorn central_server:app --port 8000`. Terminal 2: `python edge_gateway.py`. Terminals 3–4: `python sensor_node.py --node BUOY_07 --loss 0.15 --cyclone-after 15` and `python sensor_node.py --node RADAR_COAST_A --loss 0.10 --cyclone-after 25`. Then open http://127.0.0.1:8000/status, /predict and the dashboard at http://127.0.0.1:8000. Wait until the alert turns on. Then show Packet Tracer and the successful ping from BUOY-01.
3:35–4:20	Server log, then Table 4 (problems)	Show the startup-event fix in the code, and the crash_srv.log file for the malformed-packet problem.
4:20–5:00	Cost table, division-of-work table, GitHub commit 82bcb8a	Point at your tasks; end with ‘Thank you’ on webcam. Stop the programs with Ctrl+C afterwards.

Script

0:00–0:15 Introduction

Hello, I am Laksh Ahuja, register number 24BDS0212, team number ____. Our project is “AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry”, and in this video I present my individual work for Review 2.

> *SHOW: your face, then Figure 1 (architecture).*

0:15–1:55 Design walkthrough

This is the architecture of our Review 2 prototype; my modules are highlighted. Sensor nodes send JSON packets over UDP with a node name, a sequence number, wind, temperature and pressure. Anvay's gateway marks a packet CRITICAL if wind is at least sixty kilometres per hour or pressure is at most nine hundred and ninety hectopascals, and forwards critical packets first.

My central server receives each packet on port nine thousand and two and checks the sequence number. If numbers are missing — say ten arrives and then thirteen — it calls Sreehari's interpolation

function to fill eleven and twelve and marks them as reconstructed. Then it computes a risk score from zero to one hundred, sixty percent from wind and forty percent from pressure, stores the last fifty readings per node, and serves everything through a REST API and a dashboard. The alert turns on when any node's risk reaches sixty.

> *SHOW: Figure 2 (my flowchart), the parameter table, then central_server.py.*

For my design choices: I compared Flask, Django and FastAPI on cost, fit, availability, power and ease of use, and chose FastAPI because its startup hook runs the UDP listener cleanly. I chose UDP over TCP because TCP's retransmissions would hide the very loss we study, and two-second polling for the dashboard as the simplest option for now, with WebSocket planned next.

> *SHOW: Table 3 (comparison of alternatives).*

1:55–3:35 Live progress — my work

Now the real code. I start my server, the gateway, and two sensor nodes.

> *SHOW: start the four terminals.*

The sensors begin calm, then the simulated storm starts. Here is the status endpoint: packets received, packets reconstructed after loss, and critical packets. In our first twelve-second team test the server received sixty-six packets, rebuilt eleven and counted two critical, with the risk at one hundred and the alert on. In my sixty-second re-run, which is in the report, it reached a hundred and sixty received and twenty-two rebuilt, and the alert came on at about thirty seconds.

> *SHOW: /status, /predict, then the dashboard.*

On the dashboard, critical nodes are shown in red and reconstructed values in yellow, so you can see which data was estimated.

We also built the routed network in Cisco Packet Tracer with OSPF; a ping from BUOY-01 to the prediction server succeeds end to end.

> *SHOW: Packet Tracer topology and the ping.*

I am about forty percent through my planned scope. Still to do: priority scheduling on a bandwidth-limited link, real-time updates, a realistic cyclone model and automated tests.

3:35–4:20 Problems and how I adapted

First problem: the predict endpoint returned empty data at the start. The server log showed the listener starting after the sensors had begun sending. I moved it into FastAPI's startup event and it was fixed. Second, processes launched from separate shells failed silently, so we now start everything from one script.

A problem I have not fixed yet: when I sent one malformed packet in testing, the listener thread crashed and the status stayed at zero. Input validation is my first task for Review 3.

> *SHOW: the startup-event code, crash_srv.log and Table 4.*

4:20–5:00 Cost and my contribution

The cost of my part is still zero rupees; a small cloud server for a public demo would cost about five to ten dollars a month. Socially, coastal communities with the worst connectivity benefit most from a server that copes with missing data.

My contribution is the central server, the risk score, the REST API, the dashboard, the startup fix and our end-to-end test runs, in commit 82bcb8a. Thank you.

> *SHOW: the division-of-work table and the commit on GitHub.*

3. Review 3 — Final Prototype, Report & Viva

Date: 9/10/2026 **Spoken length:** 539 words ≈ 4.0 minutes **File name:** LakshAhuja_R3_Video.mp4

Before you record

1. Switch back to the current branch in the main repository folder and run `pip install -r requirements.txt`.
2. Do a practice run of `python run_demo.py` once, then close it so port 8000 is free.
3. Open results/REPORT.md (or the QoS chart in your R3 report) and the R3 comparison table.
4. Have a second terminal ready for `python -m pytest -v`.

What to show, step by step

Time	On screen	What you do
0:00–0:15	Webcam, then the terminal	Introduce yourself.
0:15–2:15	Terminal → dashboard at http://127.0.0.1:8000 (one take, no cuts)	Run `python run_demo.py`. Open the dashboard. Show calm → cyclone approaching → red alert banner. Point at yellow dots (rebuilt readings), hollow dots (late-packet predictions), the satellite panel and finally zoom into the QoS table.
2:15–4:00	QoS results table/chart, then the pytest terminal	Point at the 1200 B/s rows (356 ms vs 49 s) and the 1000 B/s rows (100 % vs 77.3 % critical delivered). Run `python -m pytest -v` and let '28 passed' appear.
4:00–4:30	R3 comparison-with-literature table	Point at each row as you speak.
4:30–5:00	Your files in the editor (central_server.py, qos.py, dashboard.html, run_demo.py, ocean_model.py, tests/) and your commits	End with 'Thank you' on webcam. Stop the demo with Ctrl+C.

Script

0:00–0:15 Introduction

Hello, I am Laksh Ahuja, register number 24BDS0212, team number ____. Our project is "AI-Driven Edge–Cloud Network for Cyclone Prediction and Resilient Ocean Telemetry", and in this video I present my individual work for Review 3.

> *SHOW: your face, then the terminal.*

0:15–2:15 Live demonstration (record without cuts)

I will now run the complete system with one command: python run_demo.py. It starts my central server, the edge gateway, three sensor nodes and a satellite ground station.

> *SHOW: run the command, open http://127.0.0.1:8000.*

At the start everything is green: normal conditions, low risk, all traffic NORMAL. The sensors lose about twelve percent of their packets in bursts, like rain fade at sea.

Now the simulated cyclone from my ocean model is approaching. Pressure falls and wind rises in the charts. Packets turn CRITICAL, and some are flagged by the machine-learning classifier even before the wind crosses sixty.

Now the banner turns red — cyclone alert. The fused risk combines the sensors, the satellite and the forecast. Yellow dots are lost packets rebuilt by the LSTM model, and hollow dots are real-time

predictions while a packet is late.

On the right is the satellite panel. Each image is about two megabytes, but only about one kilobyte of features is sent — ninety-nine point nine five percent less data.

> *SHOW: zoom into the QoS table.*

This table comes from my QoS scheduler, running inside the gateway: critical packets wait only a few hundred milliseconds, while normal and unimportant traffic waits longer.

2:15–4:00 Test results and failures fixed

My benchmark was: under congestion, ninety-five percent of critical packets must arrive within one second, with at least ninety-nine percent delivered. I tested it with three hundred seconds of traffic on links of one to two kilobytes per second.

At twelve hundred bytes per second, ninety-five percent of critical packets arrived within three hundred and fifty-six milliseconds with my scheduler, compared with forty-nine seconds for a normal first-come-first-served queue. On the tightest link, first-come-first-served delivered only seventy-seven percent of critical packets; mine delivered all of them. When the link has spare capacity, both behave the same.

> *SHOW: the QoS results table.*

In a seventy-five-second live run, the server rebuilt twenty-six lost readings, and all twenty-eight automated tests pass.

> *SHOW: run pytest and show 28 passed.*

Two things failed along the way. My evaluation hung, because two scheduler functions used slightly different rules; I made them share one rule and added a test. And capping normal traffic at fifty percent during an alert actually made critical packets slower, so I replaced it with a small reserve of bandwidth for critical packets.

4:00–4:30 Comparison with literature

Compared with DiffServ, our priority follows the live cyclone risk instead of fixed classes. Compared with RED, we drop only unimportant data, and only when the link is congested. Compared with edge-computing papers that describe the idea, we built and measured a working prototype. Our limits: the data is simulated, and the scheduler has been tested on one gateway.

> *SHOW: the comparison table.*

4:30–5:00 My contribution and scalability

My contribution is the central server, the live dashboard, the QoS scheduler, the cyclone model, the demo launcher and the test suite.

To deploy it, we would run the server in the cloud, gateways on Raspberry Pis at coastal sites, secure the API, and test with real buoy data from INCOIS. Each sensor node costs about eight hundred rupees, which makes dense coastal networks affordable. Thank you.

> *SHOW: your files and commits.*